⚠ **TRAINING EXAMPLE — NOT A REAL IEC 62304 DELIVERABLE**

This project is a **training site, not a real medical device** under any regulatory definition — it is not Software as a Medical Device (SaMD) or Software in a Medical Device (SiMD). This page is not a genuine IEC 62304 deliverable: it illustrates the *kind* of document a real SaMD/SiMD project would produce. See the document register for the full explanation.

Status: Partial — see Gaps. Clause: 5.4 — Software Detailed Design · **Register:** Document Register

What the standard requires (Class C — every sub-clause of 5.4)

REF	TITLE	APPLIES AT	OUTPUT
5.4.1	Subdivide software into software units	**B, C**	(No separate `output` — this is the one requirement of 5.4 that reaches Class B; §5.4.2–5.4.4 are Class C only)
5.4.2	Detailed design for each software unit	**C only**	Design documented in enough detail to allow correct implementation of each software unit
5.4.3	Detailed design for interfaces	**C only**	Design documented for the interfaces of each software unit, in enough detail to implement them correctly
5.4.4	Verify detailed design	**C only**	Documented verification that the detailed design implements the architecture and does not contradict it

5.4.1 — Subdividing into software units

The software items in 04 — Architecture are already at file granularity (one script, one responsibility) — for a project this size, "software unit" and "software item" largely coincide: `learn.js` is not further subdivided into separately-versioned files, but it *is* organised internally into named functions with single responsibilities (`mergeApplicability()`, `deliverablesFor()`, `classify()`, and so on), which is the practical unit boundary a detailed design document would describe.

5.4.2–5.4.3 — Detailed design, where it exists

`learn.js` has a real detailed design document: `learn_pseudocode.md`. It walks the data shape, state (`studiedPhases`, the current filter, the current level), and the logic of every function in the file, including the ones this document set leans on elsewhere — `mergeApplicability()`'s cross-file invariant check (see 11) and `deliverablesFor()`'s derivation of the deliverables list (see 03, REQ-14). This is the one file in the project where 5.4.2–5.4.3 are actually satisfied, not just described in outline.

5.4.4 — Verifying the detailed design

For `learn.js`: the pseudocode's description of `mergeApplicability()`'s behaviour matches the `applicability` test group's assertions line for line (compare `learn_pseudocode.md` against `tests/test_site.py`'s "the site refuses to render on contradictory data" checks) — that agreement is the verification evidence.

Gaps

`quiz.js`, `contact.js`, `theme.js`, `nav.js`, and `async-utils.js` have no equivalent pseudocode document. Their design exists only as inline comments and the code itself, plus the behavioural description in the project's design notes. For a real Class C project this would be a finding: 5.4.2 requires detailed design for *each* software unit, not the one that happened to get written down first.

This is deliberately left as an open gap rather than backfilled with a rushed pseudocode file for the remaining five scripts, because a document written to

close a gap rather than to be used is exactly the kind of paperwork IEC 62304 gets criticised for producing. If and when one of those files is next substantially changed, writing its detailed design *as part of that change* — the way `learn_pseudocode.md` was — would produce a more useful document than writing all five today for their own sake. Recorded as a problem in 13 — Problem Resolution's terms: identified, not yet actioned, rationale for the delay stated above.